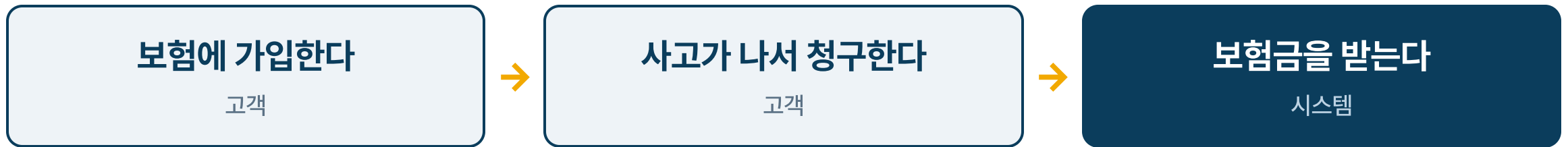


설계대로 구현하고, AI를 통제하다

의료·자동차 보험 시스템

어떤 서비스인가



고객이 보험에 가입하고, 사고가 나면 보험금을 받는 서비스입니다.

두 가지 보험을 다룬다

의료보험

진료비를 청구합니다.

입원·통원 보장, 보장 한도와 자기부담금.

청구가 간단하면 즉시 지급, 복잡하면 심사로 넘어갑니다.

자동차보험

사고를 접수합니다.

차량 정보, 운전자 범위, 사고 이력으로 보험료 산정.

접수되면 담당 직원이 지급액을 사정합니다.

세 주체가 참여한다

고객

가입자.

가입·납부·청구·사고접수.

보험사 직원

심사자.

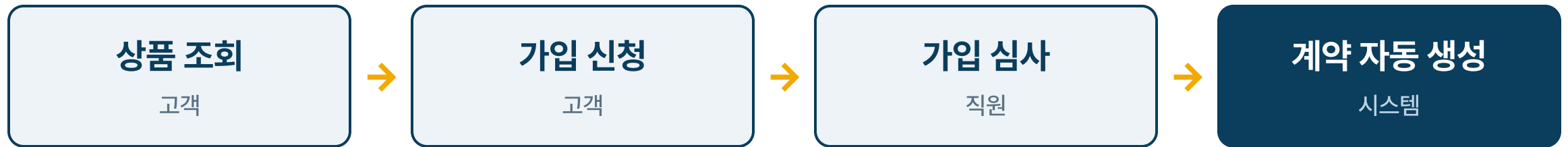
가입 심사·지급 심사.

시스템

자동 처리.

계약 생성·고지서·보험금 지급.

여정 ① — 가입



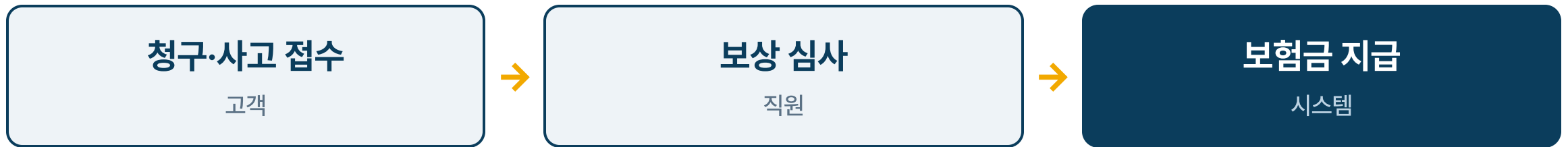
심사가 승인되면 계약이 자동으로 만들어집니다.

여정 ② — 납부와 미납



납부가 밀리면 시스템이 미납 고지서를 자동으로 보냅니다.

여정 ③ — 보상과 지급



의료청구는 복잡도에 따라, 자동차사고는 직원 사정으로 지급됩니다.

무엇으로 이루어져 있나 — 도메인

✓ 사용자

✓ 상품

✓ 계약·납부

✓ 청구·사고

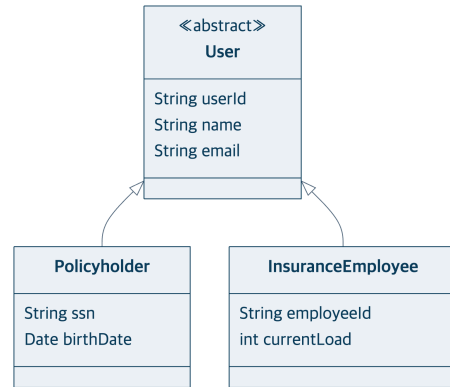
✓ 심사·지급

이 다섯 도메인을 직접 클래스 다이어그램으로 설계했습니다.

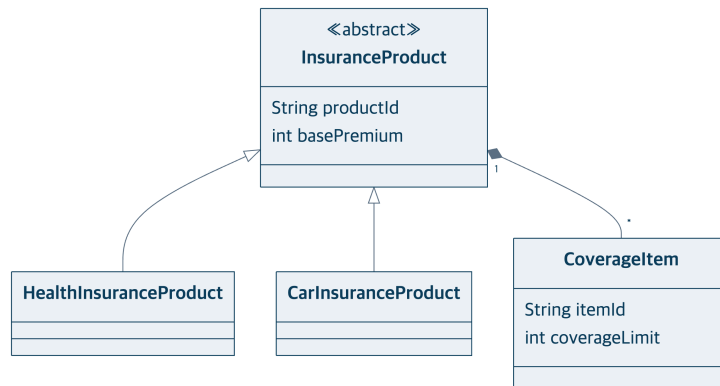
설계 = 구현 ① — 사용자 · 상품

내가 설계

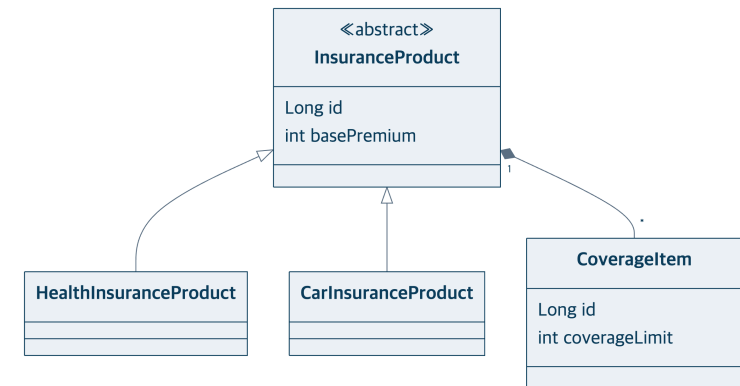
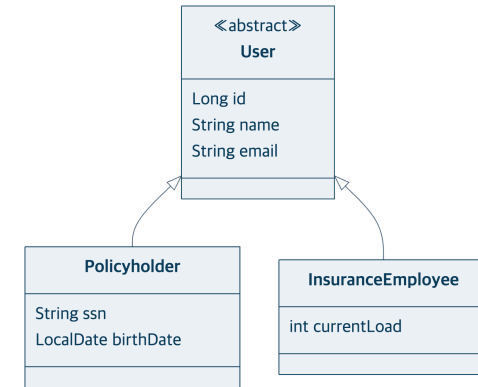
사용자



상품



코드 역설계



= 같은 점

상속 구조와 관계가 그대로입니다.

≠ 다른 점

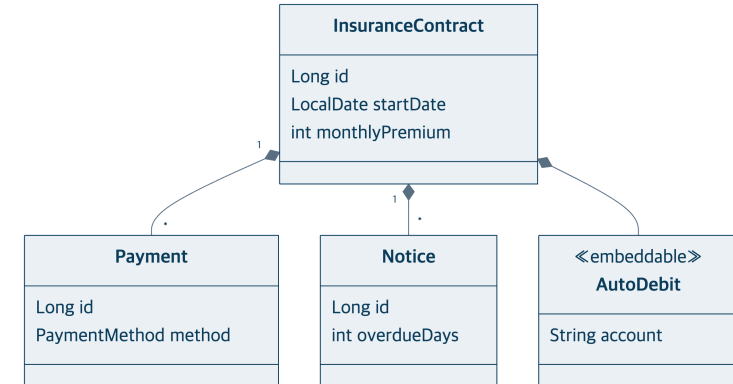
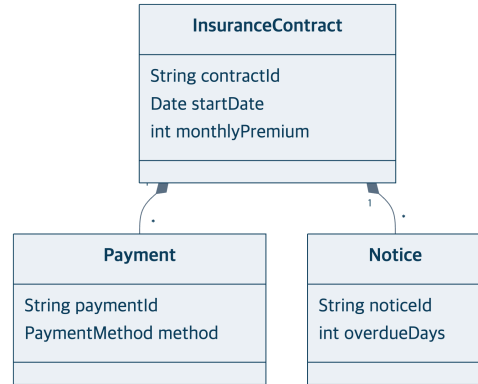
식별자 String→Long, 날짜 Date→LocalDate (JPA 표준).

설계 = 구현 ② — 계약 · 청구

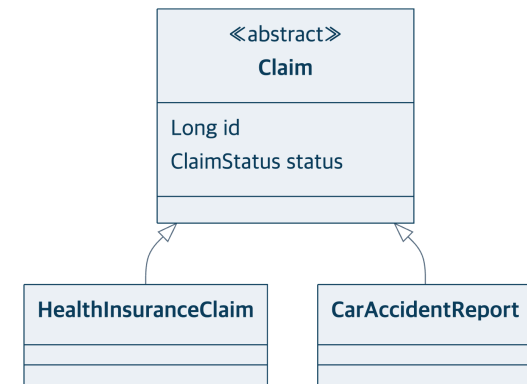
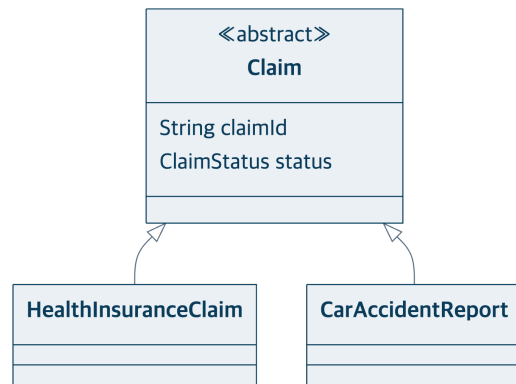
내가 설계

코드 역설계

계약



청구



= 같은 점

계약-납부-고지 구성, 청구 상속이 동일합니다.

≠ 다른 점

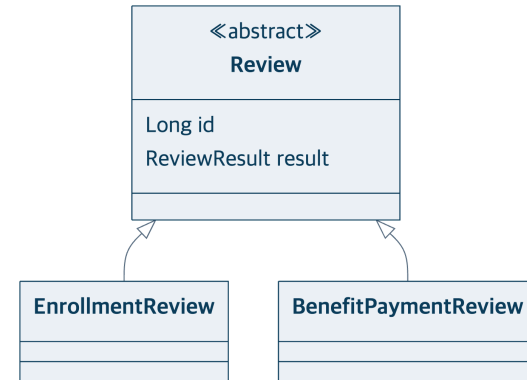
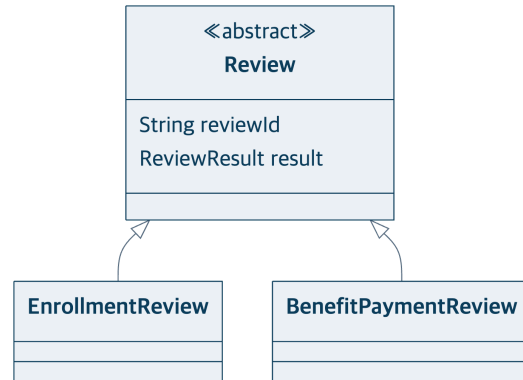
계약에 AutoDebit(자동이체) 추가, 청구 상태 4→6값.

설계 = 구현 ③ — 심사 · 사고이력

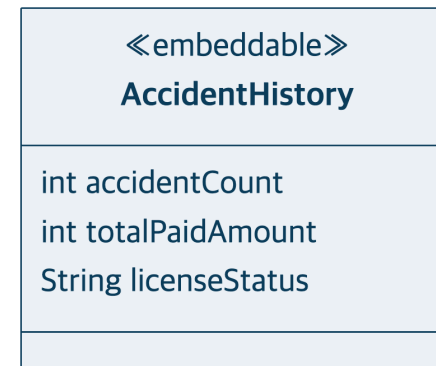
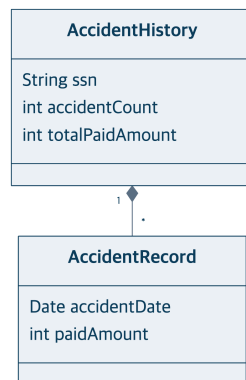
내가 설계

코드 역설계

심사



사고이력



= 같은 점

심사 상속 구조가 동일합니다.

≠ 다른 점

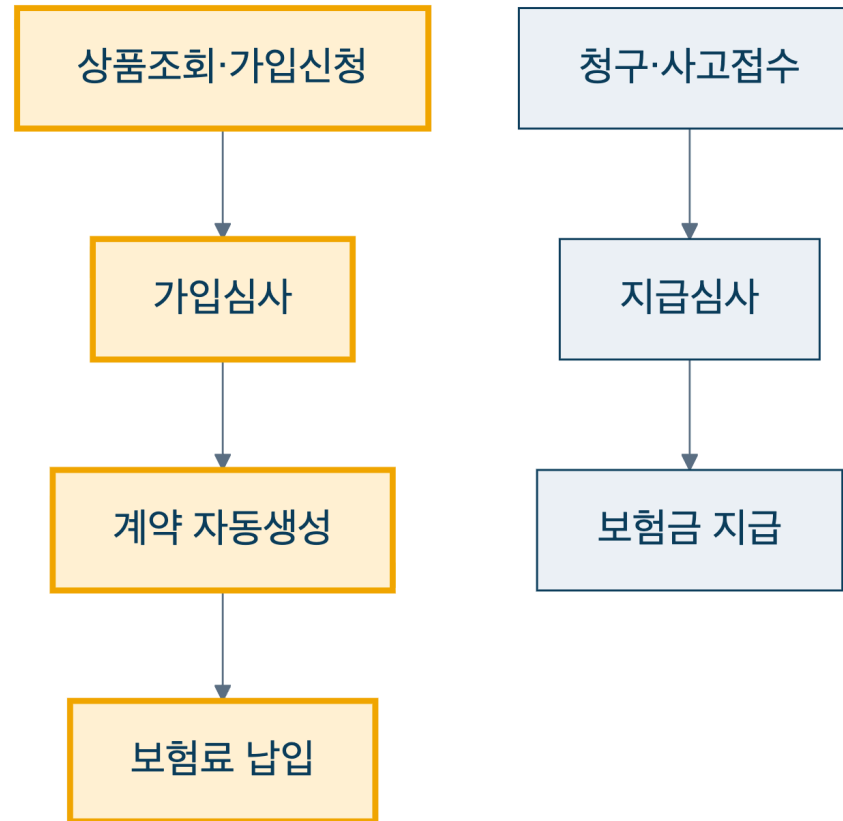
사고이력은 더미라 AccidentRecord 제거, 지급심사 대상에 자동차 포함.

차이는 왜 생겼나

차이	이유
식별자 String → Long , 날짜 → LocalDate	JPA 표준 매핑
계약에 AutoDebit 추가	자동이체 기능
청구 상태 4값 → 6값	송금 성공·실패까지 추적 (ADR 0007)
지급심사 대상 → 자동차사고 포함	자동차도 사정 필요 (ADR 0009)
AccidentRecord 제거	더미 외부연동 단순화

차이는 모두 구현상의 이유가 있고, ADR로 기록했습니다.

유스케이스도 설계대로 동작한다



설계한 유스케이스가 데모에서 그대로 동작합니다 (노란 경로).

AI를 어떻게 썼는가

사람(설계자)

다이어그램 설계 · 결정(ADR) · 리뷰

AI

구현 · 테스트 · 리팩토링

설계

다이어그램



취조

모델 대조



계획



TDD 구현

AI



리뷰

설계와 결정은 사람이, 구현은 AI가 했습니다.

AI를 쓰며 생긴 문제와 해결

① 성급한 추상화

실익 없는 계층 분리 → 되돌림

② 명세와 불일치

자동 배정 → 수동 배정으로 정정

③ 계층 패턴 이탈

점검으로 식별·관리

④ 용어·도메인 오염

용어집·ADR로 사전 차단

AI는 빠르지만, 방향은 사람이 잡았습니다.

결론

설계한 그대로 구현했고, 그 과정에서 AI를 통제했습니다.

감사합니다.